

PPO & DPO 리마인드

GRPO를 이해하기 위한 사전 지식 정리

Proximal Policy Optimization과 Direct Preference Optimization

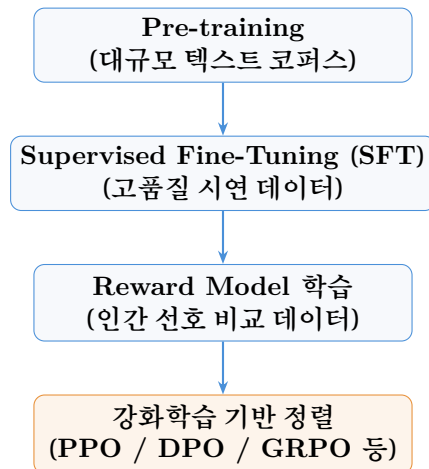
HAI 논문리딩반 - 5조

Contents

1	LLM 정렬(Alignment) 개요	2
2	PPO (Proximal Policy Optimization)	3
2.1	배경: 정책 경사(Policy Gradient)와 TRPO	3
2.2	PPO의 핵심 아이디어	3
2.2.1	PPO-Clip 목적 함수	3
2.2.2	클리핑의 직관적 이해	4
2.3	Advantage 추정: GAE	4
2.4	LLM에서의 PPO (RLHF)	4
2.5	PPO의 장단점	5
3	DPO (Direct Preference Optimization)	6
3.1	동기: 보상 모델 없이 정렬할 수 있을까?	6
3.2	Bradley-Terry 모델과 DPO 손실 함수	6
3.3	DPO의 직관적 이해	7
3.4	DPO의 장단점	7
4	PPO vs DPO 종합 비교	9
4.1	구조적 비교	9
4.2	상세 비교표	9
4.3	언제 무엇을 쓸까?	9
5	GRPO로의 연결	10

1 LLM 정렬(Alignment) 개요

대형 언어 모델(LLM)은 방대한 텍스트 코퍼스로 사전 학습(Pre-training)된 후, 인간의 의도와 선호에 맞게 정렬(Alignment)되어야 한다. 이 정렬 과정은 일반적으로 다음의 파이프라인을 따른다:



왜 정렬이 필요한가?

사전 학습된 LLM은 “다음 토큰 예측”이라는 목적 함수로 학습되므로, 유해한 콘텐츠 생성, 사실과 다른 답변(hallucination), 사용자 의도 무시 등의 문제가 발생할 수 있다. 정렬은 모델이 도움이 되고(helpful), 정직하며(honest), 무해한(harmless) 응답을 생성하도록 유도하는 과정이다.

RLHF (Reinforcement Learning from Human Feedback)

InstructGPT(Ouyang et al., 2022)에서 체계화된 RLHF 파이프라인은 다음 3단계로 구성된다:

1. **SFT**: 인간이 작성한 시연(demonstration) 데이터로 지도 학습
 2. **Reward Model**: 인간의 선호 비교 데이터 (x, y_w, y_l) 로 보상 모델 학습
 3. **RL Fine-tuning**: 보상 모델의 피드백으로 정책(Policy) 최적화 → **PPO**가 대표적
- DPO는 이 중 2단계와 3단계를 하나로 합쳐 보상 모델 없이 직접 정렬하는 방법이다.

2 PPO (Proximal Policy Optimization)

2.1 배경: 정책 경사(Policy Gradient)와 TRPO

강화학습에서 정책 π_θ 를 직접 최적화하는 정책 경사(Policy Gradient) 방법은 다음의 목적 함수를 최대화한다:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

기본 정책 경사(REINFORCE)는 분산이 크고 학습이 불안정하다는 문제가 있다. 이를 해결하기 위해 **TRPO (Trust Region Policy Optimization)**가 제안되었으나, KL 발산 제약 조건을 다루기 위한 2차 최적화(conjugate gradient)가 필요하여 구현이 복잡하다.

TRPO의 핵심 아이디어

정책 업데이트 시 이전 정책과 새 정책 사이의 KL 발산을 일정 범위(δ) 이내로 제한한다:

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \quad \text{s.t.} \quad \mathbb{E}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_\theta(\cdot|s_t)]] \leq \delta$$

이 제약 조건 최적화는 효과적이지만 구현이 복잡하고 계산 비용이 높다.

2.2 PPO의 핵심 아이디어

PPO(Schulman et al., 2017)는 TRPO의 성능을 유지하면서 구현을 대폭 단순화한 알고리즘이다. 핵심은 **클리핑(Clipping)**을 통해 정책 업데이트의 크기를 제한하는 것이다.

2.2.1 PPO-Clip 목적 함수

PPO-Clip 목적 함수

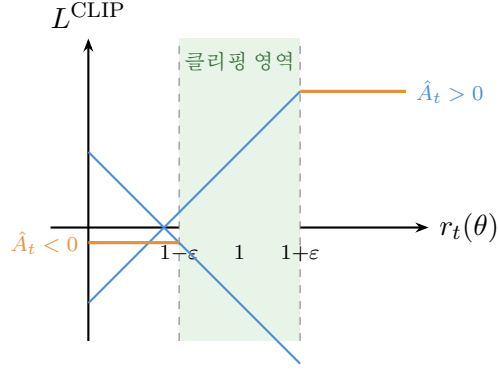
$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right] \quad (1)$$

여기서 확률비(probability ratio)는:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

- $\hat{A}_t$: 시점 t 에서의 Advantage 추정값
- ε : 클리핑 범위 (보통 0.1-0.2)
- $\text{clip}(r, 1-\varepsilon, 1+\varepsilon)$: r 을 $[1-\varepsilon, 1+\varepsilon]$ 범위로 잘라냄

2.2.2 클리핑의 직관적 이해



클리핑이 하는 일

- $\hat{A}_t > 0$ (좋은 행동): 확률비 r_t 가 $1+\epsilon$ 을 넘으면 더 이상 목적 함수가 증가하지 않는다. → 좋은 행동의 확률을 과도하게 높이는 것을 방지
- $\hat{A}_t < 0$ (나쁜 행동): 확률비 r_t 가 $1-\epsilon$ 아래로 내려가면 더 이상 목적 함수가 감소하지 않는다. → 나쁜 행동의 확률을 과도하게 낮추는 것을 방지
- 결과적으로 정책이 한 번의 업데이트에서 너무 크게 변하지 않도록 “신뢰 영역(Trust Region)”을 암묵적으로 형성한다.

2.3 Advantage 추정: GAE

PPO에서 Advantage $\hat{A}_t$ 는 **GAE (Generalized Advantage Estimation)**로 추정한다:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

여기서 TD 잔차(temporal difference residual)는:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

GAE의 편향-분산 트레이드오프

- $\lambda = 0$: $\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \rightarrow$ 낮은 분산, 높은 편향 (1-step TD)
- $\lambda = 1$: $\hat{A}_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t) \rightarrow$ 높은 분산, 낮은 편향 (Monte Carlo)
- 실전에서는 $\lambda \in [0.9, 0.99]$ 로 설정하여 적절한 균형을 맞춘다.

2.4 LLM에서의 PPO (RLHF)

LLM 정렬에서 PPO를 적용할 때는 일반 RL과 다른 특수한 구조를 갖는다:

RLHF에서의 PPO 구성 요소 (4개 모델)

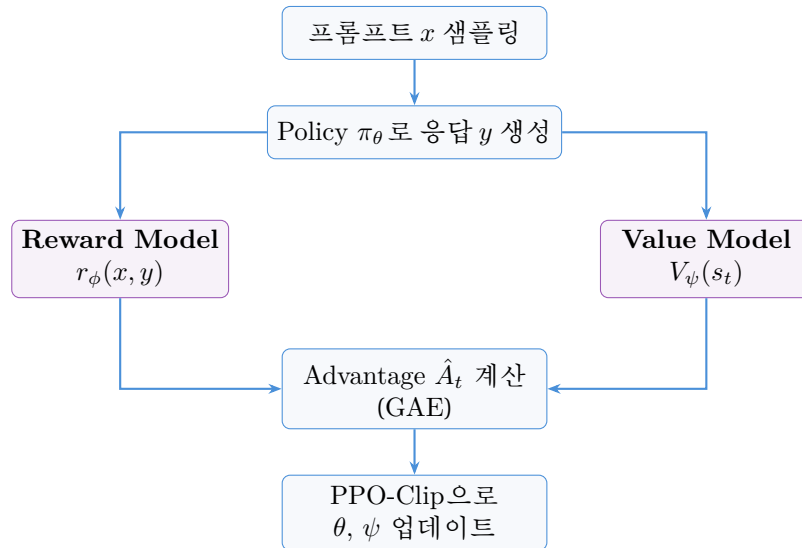
1. **Policy Model** π_θ : 현재 학습 중인 LLM (응답 생성)
2. **Reference Model** π_{ref} : SFT 직후의 고정된 LLM (KL 페널티 기준)
3. **Reward Model** r_ϕ : 인간 선호 데이터로 학습된 보상 모델
4. **Value Model (Critic)** V_ψ : 상태 가치를 추정하는 모델 (Advantage 계산용)

RLHF에서의 PPO 목적 함수는 다음과 같다:

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} \left[r_{\phi}(x, y) - \beta \text{KL}[\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)] \right]$$

KL 페널티의 역할

KL 페널티 $\beta \text{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}]$ 는 정책 모델이 기준 모델에서 너무 멀어지는 것을 방지한다. 이것이 없으면 모델은 보상 모델의 허점을 악용하여 **보상 해킹(Reward Hacking)**을 하게 된다: 높은 보상 점수를 받지만 실제로는 품질이 낮은 응답을 생성하는 현상이다.



2.5 PPO의 장단점

PPO 장점

- 안정적인 학습: 클리핑으로 급격한 정책 변화를 방지
- 범용성: 다양한 보상 함수와 태스크에 적용 가능
- TRPO 대비 구현이 단순하고 계산 효율적
- 온라인 학습: 현재 정책에서 새로운 데이터를 생성하여 학습

PPO 단점 (특히 LLM 정렬에서)

- 4개 모델 동시 운용: Policy, Reference, Reward, Value 모델을 모두 메모리에 올려야 하므로 GPU 메모리 부담이 매우 크다. 7B 모델 기준 약 **56GB** 이상의 GPU 메모리가 필요하다.
- 보상 모델 학습 필요: 별도의 인간 선호 데이터 수집과 보상 모델 학습 파이프라인이 필요하다.
- 하이퍼파라미터 민감성: ϵ, β , GAE의 λ 등 조정해야 할 하이퍼파라미터가 많다.
- 학습 불안정성: 보상 모델의 품질에 크게 의존하며, 보상 해킹 위험이 존재한다.
- 샘플 효율성: 온라인으로 응답을 생성해야 하므로 학습 속도가 느리다.

3 DPO (Direct Preference Optimization)

3.1 동기: 보상 모델 없이 정렬할 수 있을까?

PPO 기반 RLHF의 복잡성을 해결하기 위해, Rafailov et al. (2023)은 보상 모델을 명시적으로 학습하지 않고 인간 선호 데이터로부터 직접 정책을 최적화하는 **DPO**를 제안하였다.

DPO의 핵심 통찰

RLHF의 보상 최대화 문제:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[r(x, y)] - \beta \text{KL}[\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)]$$

의 최적해(closed-form solution)가 다음과 같다는 것을 이용한다:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

이를 $r(x, y)$ 에 대해 역으로 풀면:

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

즉, 보상 함수를 최적 정책과 기준 정책의 로그 확률비로 표현할 수 있다.

3.2 Bradley-Terry 모델과 DPO 손실 함수

인간의 선호는 **Bradley-Terry** 모델로 모델링한다. 응답 y_w 가 y_l 보다 선호될 확률은:

$$p(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$$

여기서 σ 는 시그모이드 함수이다.

보상 함수를 정책의 로그 확률비로 대체하면, $Z(x)$ 항이 상쇄되어 다음의 **DPO** 손실 함수를 얻는다:

DPO 손실 함수

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

- (x, y_w, y_l) : 프롬프트 x 에 대해 선호 응답 y_w (winner)와 비선호 응답 y_l (loser)
- π_{θ} : 학습 중인 정책 모델
- π_{ref} : 고정된 기준 모델 (SFT 모델)
- β : 기준 모델로부터의 이탈 정도를 조절하는 온도 파라미터

3.3 DPO의 직관적 이해

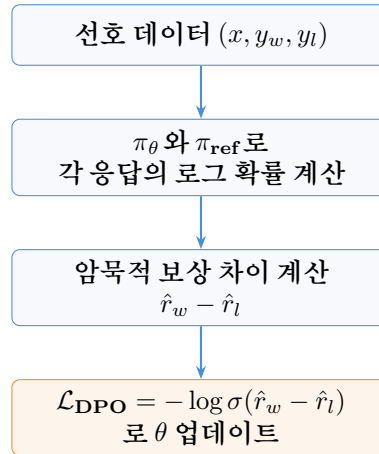
DPO 손실 함수가 하는 일

DPO 손실 함수의 그래디언트를 분석하면:

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} \propto -\beta \underbrace{\sigma(\hat{r}_l - \hat{r}_w)}_{\text{가중치}} \left[\underbrace{\nabla_{\theta} \log \pi_{\theta}(y_w|x)}_{\text{선호 응답 확률}\uparrow} - \underbrace{\nabla_{\theta} \log \pi_{\theta}(y_l|x)}_{\text{비선호 응답 확률}\downarrow} \right]$$

여기서 $\hat{r} = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ 는 **암묵적 보상(implicit reward)** 이다.

- 선호 응답 y_w 의 생성 확률을 높이고, 비선호 응답 y_l 의 생성 확률을 낮춘다.
- 가중치 $\sigma(\hat{r}_l - \hat{r}_w)$: 모델이 아직 잘못 판단하고 있는 샘플(비선호 응답에 높은 암묵적 보상을 부여하는 경우)에 더 큰 가중치를 부여한다.
- 이미 올바르게 구분하고 있는 쉬운 샘플은 자연스럽게 가중치가 줄어든다.



3.4 DPO의 장단점

DPO 장점

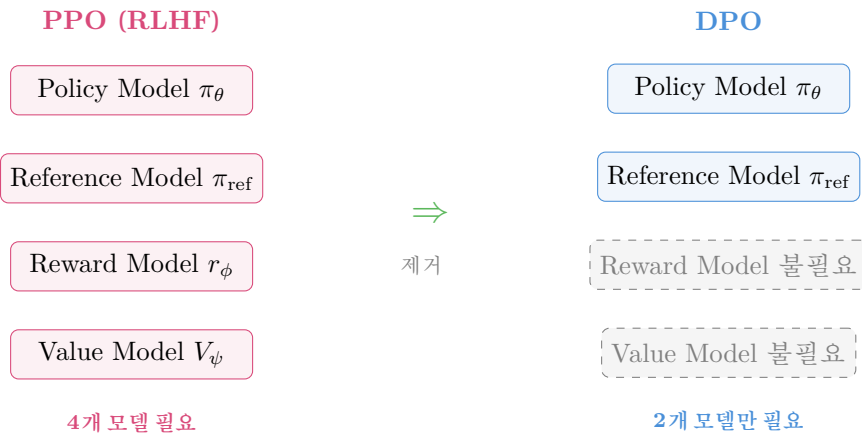
- **단순한 파이프라인**: 보상 모델 학습과 RL 루프가 불필요하다. 2개 모델(Policy + Reference)만 필요하다.
- **안정적 학습**: 지도 학습(cross-entropy)과 유사한 형태이므로 학습이 안정적이다.
- **메모리 효율**: PPO 대비 Reward Model과 Value Model이 불필요하여 GPU 메모리를 크게 절약한다.
- **이론적 보장**: RLHF 목적 함수의 최적해와 동치임이 수학적으로 증명되어 있다.
- **구현 용이**: 기존 SFT 코드에 손실 함수만 변경하면 되므로 구현이 매우 간단하다.

DPO 단점

- 오프라인 학습의 한계: 고정된 선호 데이터셋에 의존하므로, 학습 중 정책이 변해도 새로운 데이터를 생성하지 않는다. 이로 인해 **분포 이동(distribution shift)** 문제가 발생할 수 있다.
- 선호 데이터 품질 의존: 선호 데이터의 품질과 다양성에 크게 의존한다. 노이즈가 많은 데이터에 취약하다.
- 탐색(Exploration) 부재: 온라인 RL과 달리 새로운 응답을 탐색하지 않으므로, 학습 데이터에 없는 더 나은 응답을 발견하기 어렵다.
- 보상 함수의 암묵성: 명시적 보상 모델이 없으므로, 보상 함수의 품질을 직접 평가하거나 디버깅하기 어렵다.
- 수학적 추론 한계: 결과 기반 보상(정답/오답)을 직접 활용하기 어려워, 수학 문제 풀이와 같은 태스크에서는 PPO/GRPO 대비 성능이 떨어질 수 있다.

4 PPO vs DPO 종합 비교

4.1 구조적 비교



4.2 상세 비교표

항목	PPO	DPO
학습 방식	온라인 강화학습 (RL)	오프라인 지도 학습 (SL)
필요 모델 수	4개 (Policy, Ref, Reward, Value)	2개 (Policy, Ref)
보상 모델	명시적으로 학습 필요	불필요 (암묵적 보상)
학습 데이터	온라인 생성 (현재 정책에서 샘플링)	오프라인 고정 데이터셋
탐색	있음 (새로운 응답 생성)	없음 (고정 데이터)
메모리 사용	매우 높음 ($\sim 4\times$ 모델 크기)	보통 ($\sim 2\times$ 모델 크기)
구현 복잡도	높음 (RL 루프, GAE 등)	낮음 (SFT와 유사)
학습 안정성	불안정할 수 있음	비교적 안정적
하이퍼파라미터	많음 ($\epsilon, \beta, \lambda, \text{lr}$ 등)	적음 (β, lr)
수학 추론	강함 (결과 보상 직접 활용)	상대적으로 약함
분포 이동	적음 (온라인 생성)	있음 (오프라인 데이터)

4.3 언제 무엇을 쓸까?

사용 시나리오별 권장

- PPO가 적합한 경우:
 - 수학, 코딩 등 정답이 명확한 태스크 (결과 기반 보상 활용 가능)
 - 충분한 GPU 자원이 확보된 경우
 - 탐색을 통한 새로운 전략 발견이 중요한 경우
- DPO가 적합한 경우:
 - 대화, 요약 등 선호가 주관적인 태스크
 - GPU 자원이 제한적인 경우
 - 빠른 프로토타이핑이 필요한 경우
 - 고품질 선호 데이터가 충분히 확보된 경우
- GRPO가 적합한 경우:
 - PPO의 탐색 능력이 필요하지만 메모리가 부족한 경우
 - 수학 추론 등 결과 기반 보상이 가능한 태스크
 - Value Model 없이 효율적인 RL을 원하는 경우

5 GRPO로의 연결

PPO와 DPO의 한계를 이해하면 GRPO의 설계 동기가 명확해진다:

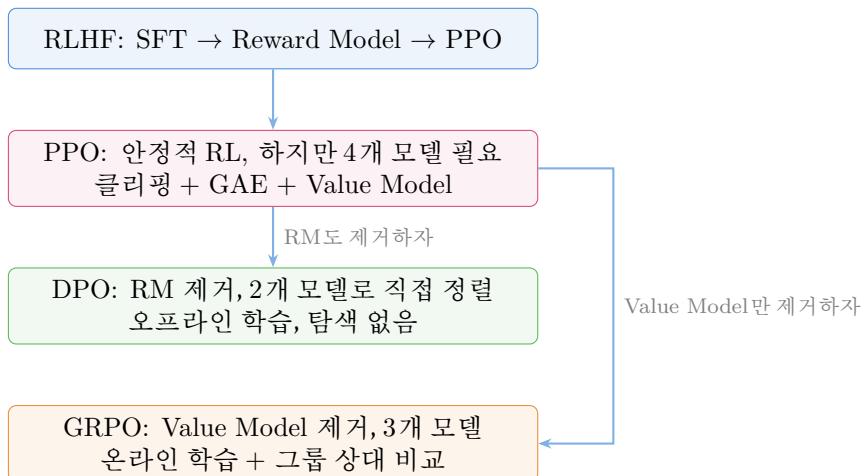
PPO → GRPO: 무엇이 바뀌었나?

요소	PPO	GRPO
Advantage 추정	$\hat{A}_t = R_t - V_\phi(s_t)$ (Value Model 필요)	$\hat{A}_i = \frac{r_i - \mu}{\sigma}$ (그룹 상대 비교)
Value Model	필요 (Policy와 동일 크기)	불필요
필요 모델 수	4개	3개
보상 활용	토큰 단위 (process-level)	출력 단위 (outcome-level)
클리핑	✓	✓ (동일)
KL 페널티	보상에 추가	손실에 직접 적용

DPO vs GRPO: 접근 방식의 차이

- **DPO**: 보상 모델 자체를 제거하고, 선호 데이터로 직접 정책을 학습한다. → 오프라인, 탐색 없음
- **GRPO**: 보상 모델은 유지하되, Value Model을 제거하고 그룹 상대 비교로 대체한다. → 온라인, 탐색 있음
- GRPO는 PPO의 “탐색 능력”과 DPO의 “효율성”을 절충한 방법으로 볼 수 있다.

Summary



전체 흐름 요약

PPO는 RLHF의 표준 알고리즘으로, 클리핑 기반의 안정적인 정책 최적화를 제공하지만 4개 모델을 동시에 운용해야 하는 메모리 부담이 크다. **DPO**는 보상 모델 없이 선호 데이터로 직접 정렬하여 파이프라인을 대폭 단순화했지만, 오프라인 학습의 한계로 탐색이 불가능하다. **GRPO**는 PPO에서 Value Model만 제거하고 그룹 상대 비교로 대체하여, 온라인 탐색 능력을 유지하면서도 메모리 효율을 크게 개선한 방법이다.